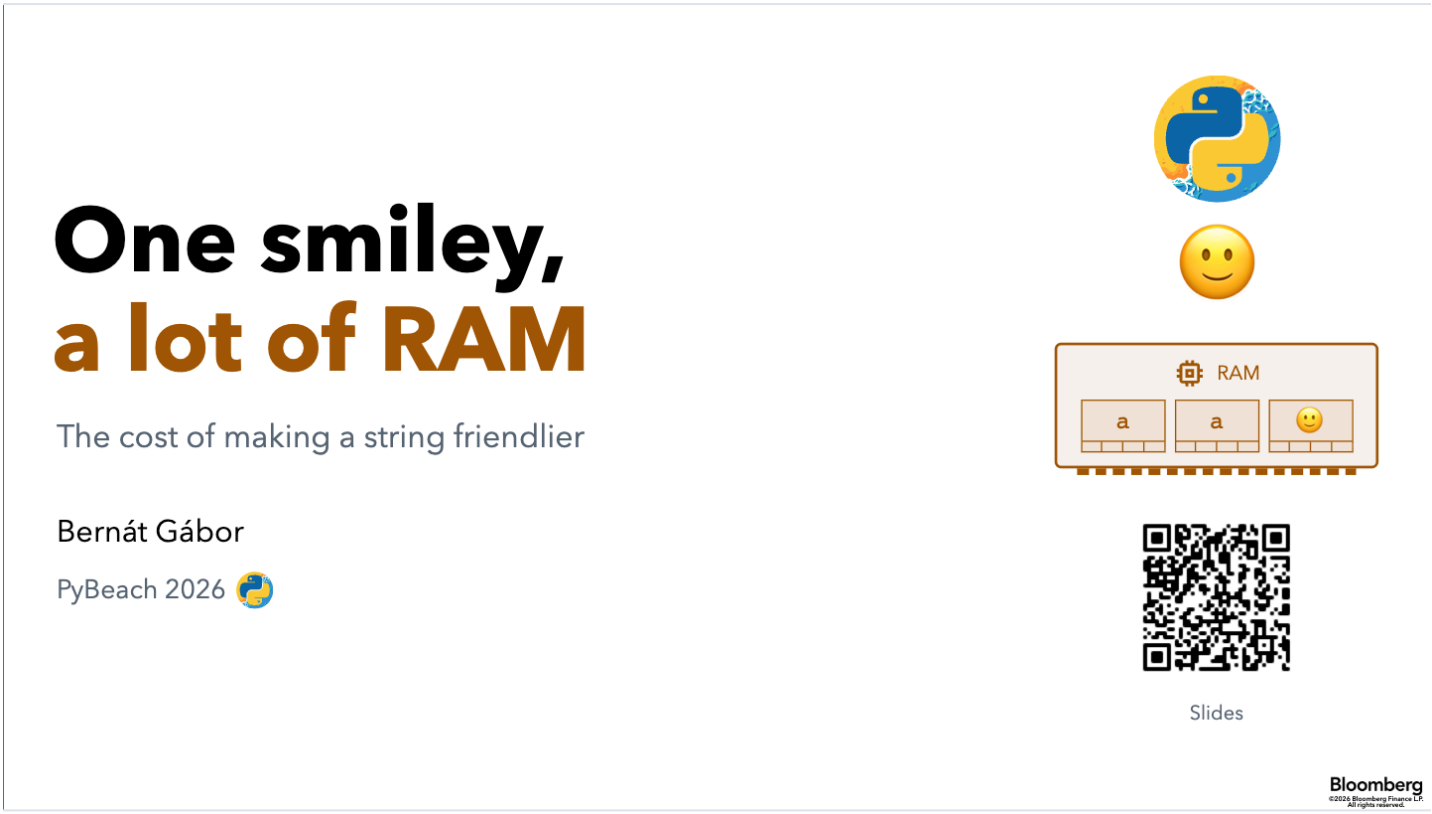



[00:00–00:15]

You know the feature request. Everything works, the tests pass, and somebody says: could we make it a little friendlier? Absolutely. One smiley. What could that possibly cost? I'm Bernát. I work on Python tools. Let's look at the memory bill.

[Pause for a beat before advancing.]




[00:15–00:55]

Here are a million lowercase "a" characters. A very boring document, but the requirements didn't say it had to be good. Python reports about one megabyte. Add one smiley. Have a guess: four extra bytes? A little object overhead? Who thinks we're about to regret this?

[Give the room three seconds. The recording holds before the result. Click when ready.]

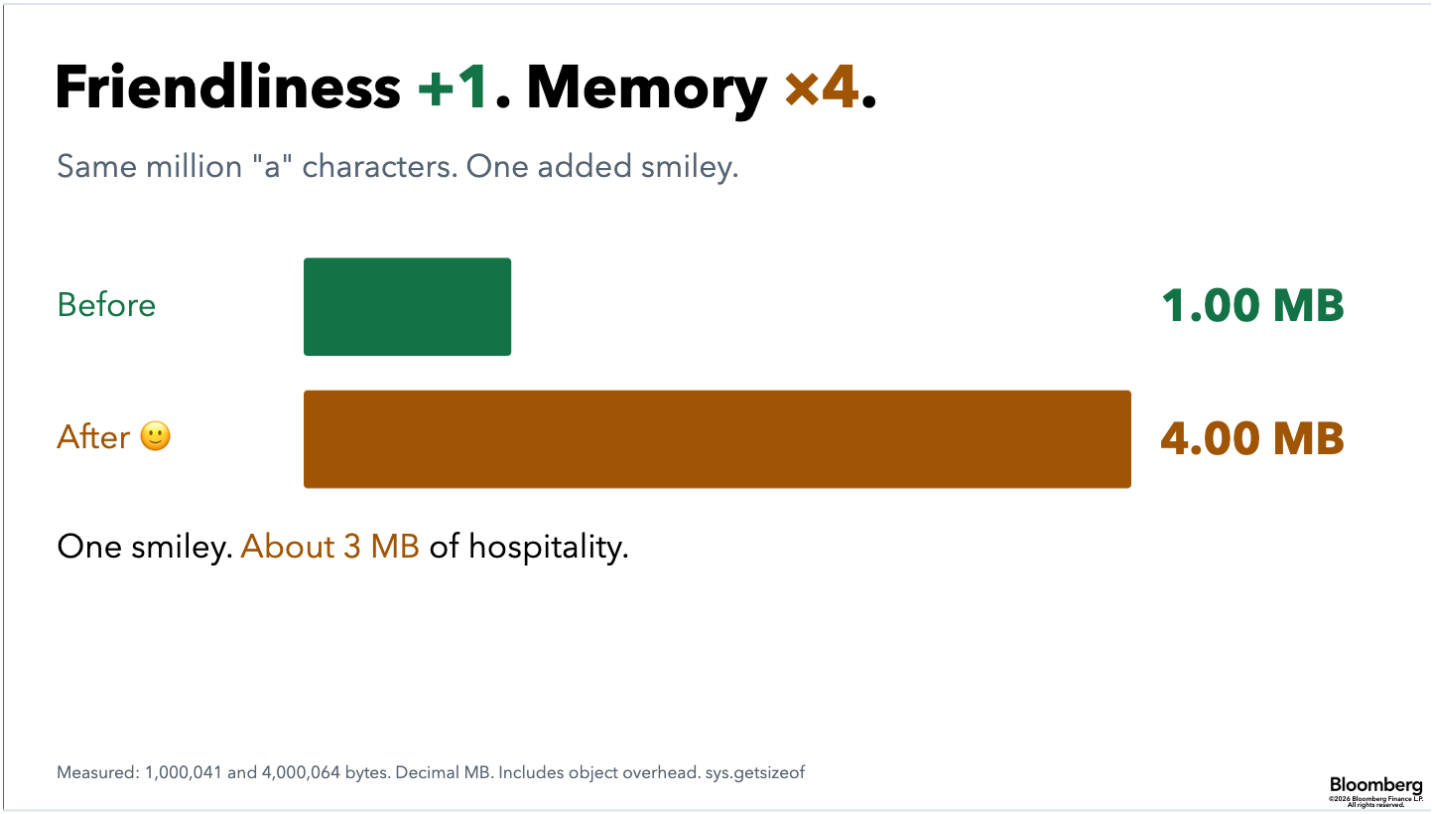
Four megabytes.

[Hold the number for a beat.]

We gave it emotional support. It asked for more RAM.

[Pause; continue when the room settles.]

That's the size of the string object itself. Something happened to all the perfectly innocent "a" characters already there.




[00:55–01:15]

Here's the itemized bill. Original text: roughly one megabyte.

[Click: the hospitality bill.]

Friendlier text: roughly four. That's about three megabytes of hospitality for one guest.

[Pause on the longer bar.] The expensive part is the storage for every character. Our new guest has changed the seating arrangement for the whole document.




[01:15–02:05]

Each row shows a different string. A Unicode code point is a number that identifies a character. CPython picks one, two, or four bytes to hold the highest code point in the string, then uses that width throughout. The letter "a" only needs one byte.

[Click: accented letter.]

So does the "á" in my name. Still flying economy. [Click: euro. Let the slots widen.]

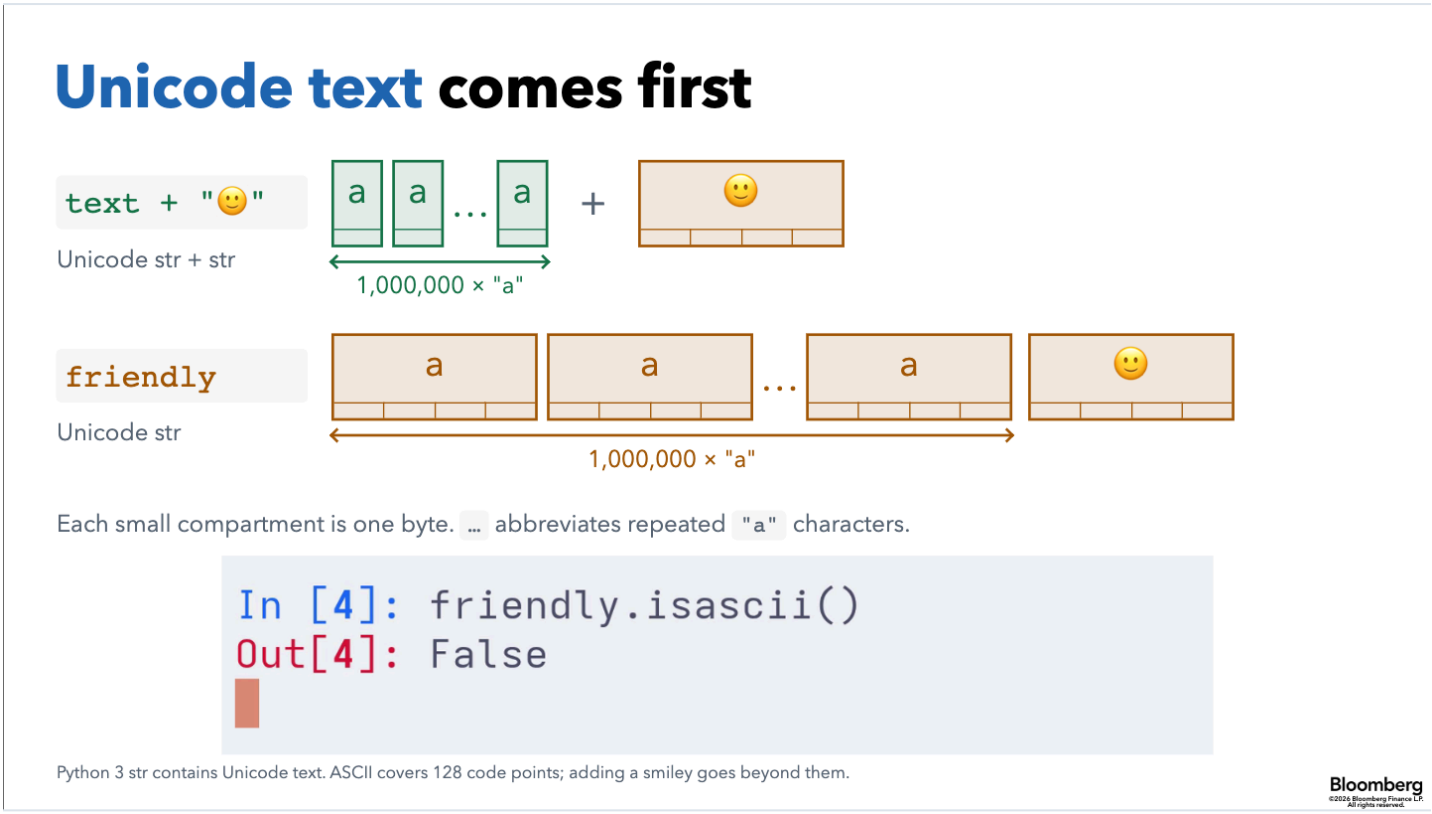
The euro sign needs two bytes. Every "a" now gets two as well.

[Click: the smiley arrives, then every slot widens. Keep the earlier rows visible.]

Our smiley needs four bytes. Every "a" gets four. One passenger needs a wider seat, so everyone gets upgraded to business class.

[Let the comparison land. Point along the four-byte row.]

Fixed-width slots let CPython locate a value by its index without walking through the earlier ones. That width applies to the whole string, including the boring passengers.

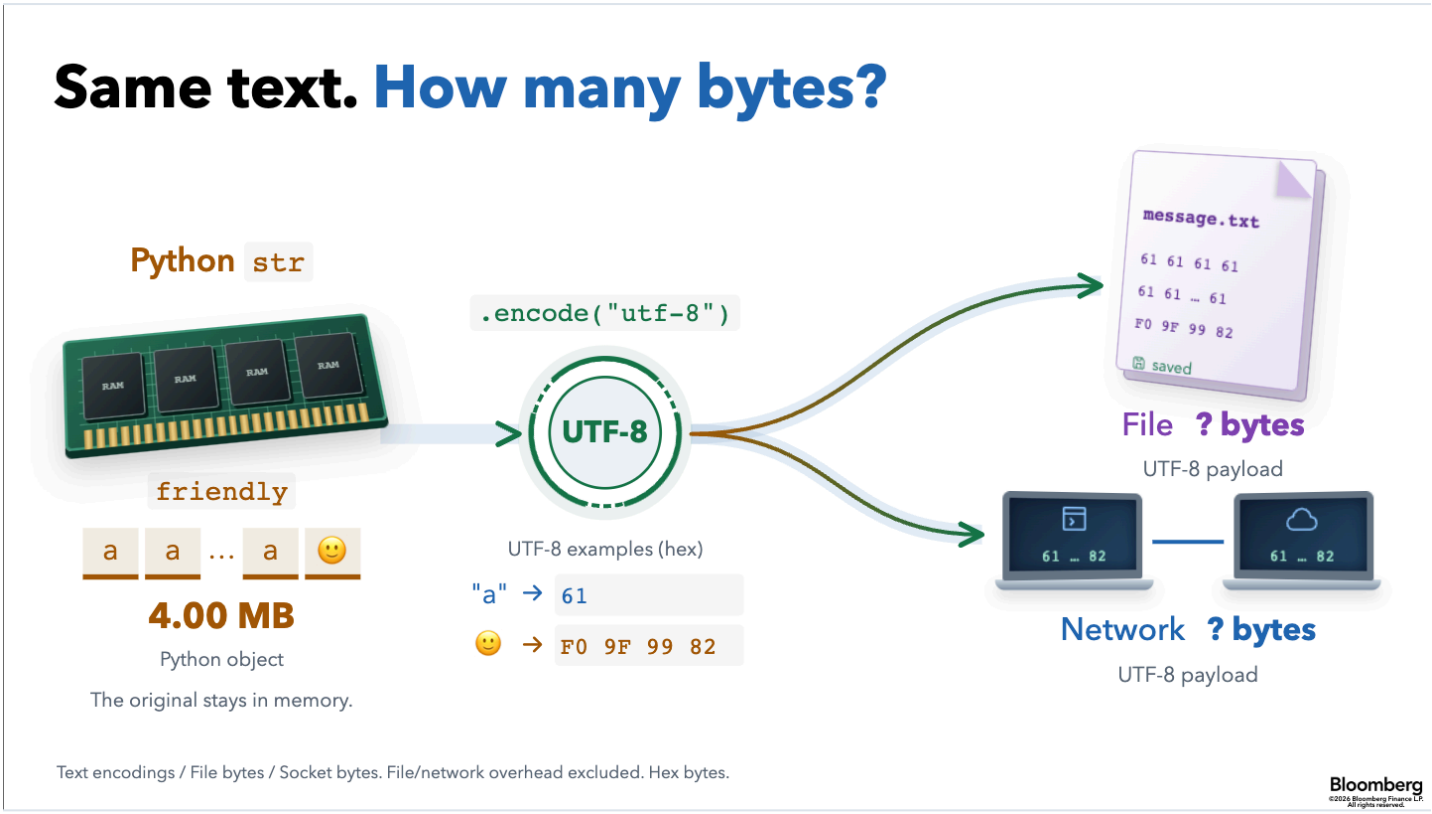



[02:05–02:25]

Both inputs are Unicode strings. The million "a" characters happen to fit in ASCII, a small character set covering basic English letters, digits, and punctuation. The smiley doesn't.

[Click: combined string and interpreter recording. The result is not ASCII-only.]

Adding them gives another Unicode string. Python picks its memory layout from the characters it contains.

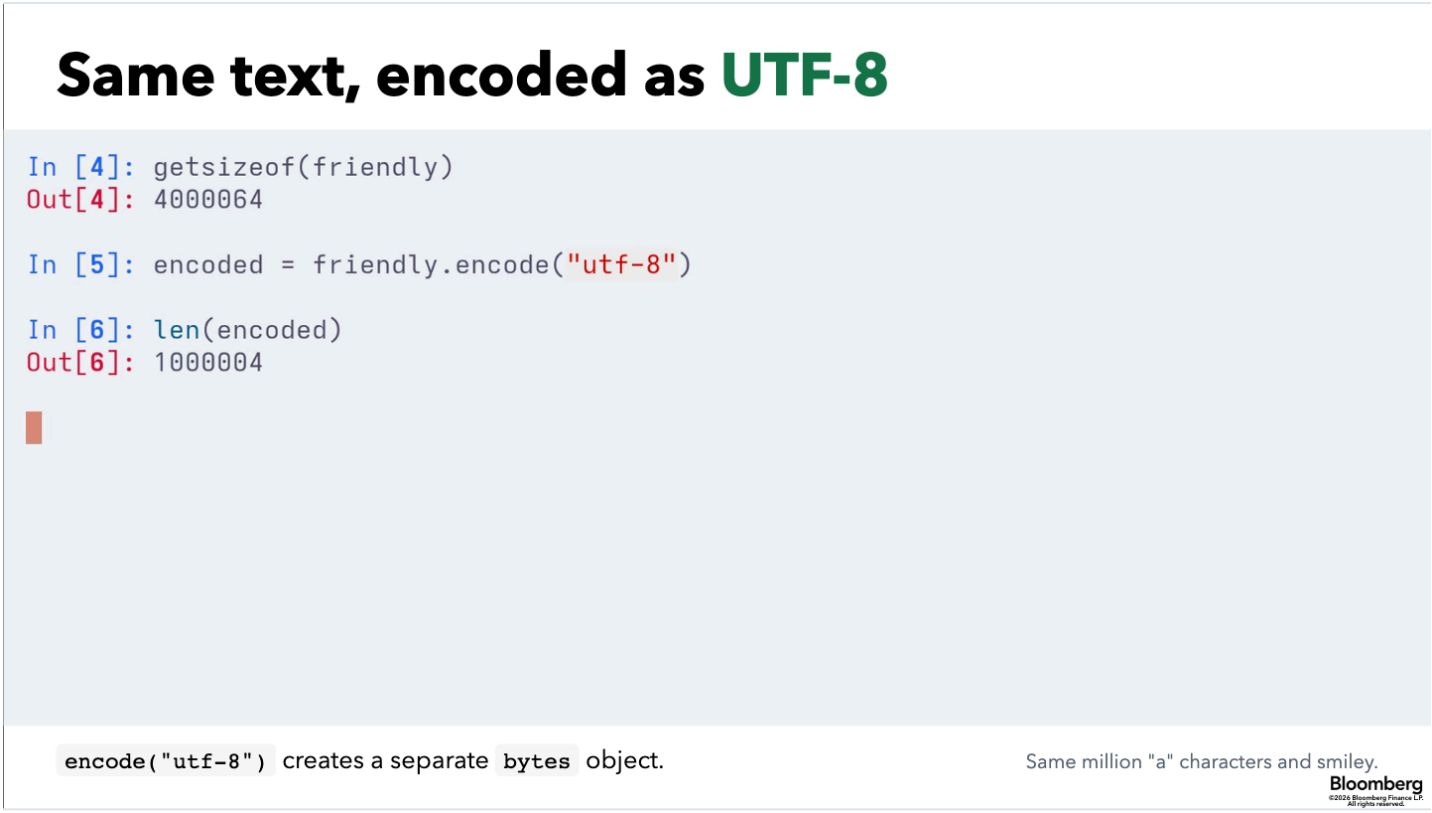



[02:25–02:35]

Files and network connections carry bytes. We choose UTF-8. Will sending this text cost four megabytes too?

[Follow the automatic animation from characters to bytes.]

Let's measure it.




[02:35–02:55]

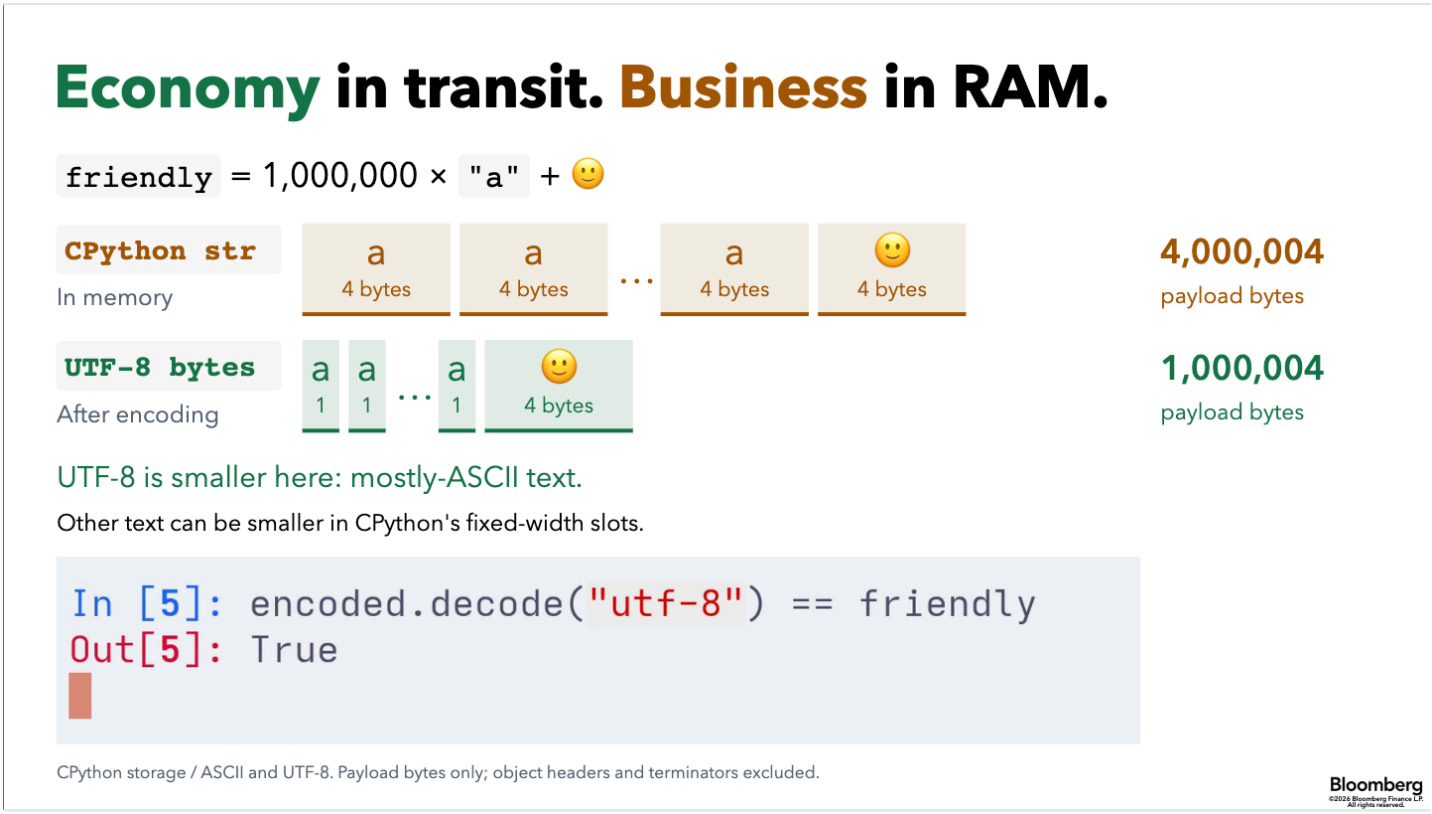
Here's the same friendly string. Get-size-of still reports about four megabytes for the Python object.

[Point to the encoding argument.]

We choose UTF-8, producing a separate bytes object. The original string stays unchanged.

[Point to the final result.]

The encoded payload is one million and four bytes, about one megabyte. We kept every character; only the byte layout changed. No compression, and nobody confiscated the smiley.




[02:55–03:25]

Same text in both rows. CPython uses four bytes for every code point, including every "a".

[Click: UTF-8 row.]

UTF-8 uses one byte for each "a" and four for this smiley. Mostly ASCII, so UTF-8 wins. Write the novel in Chinese and the maths flips.

[Click: the byte copies rise into RAM and widen; the decode recording starts. Pause on the comparison.]

About four megabytes versus one. Decoding those UTF-8 bytes restores the wider Python string. It travels economy. In RAM, it's back in business class.

[Pause; continue when the room settles.]

# Refund approved

```
In [4]: getsizeof(friendly)
Out[4]: 4000064

In [5]: plain = friendly[:-1]

In [6]: getsizeof(plain)
Out[6]: 1000041

In [7]: getsizeof(friendly)
Out[7]: 4000064
```

Slicing creates a **new string** that can use narrower storage.

The original string is unchanged.

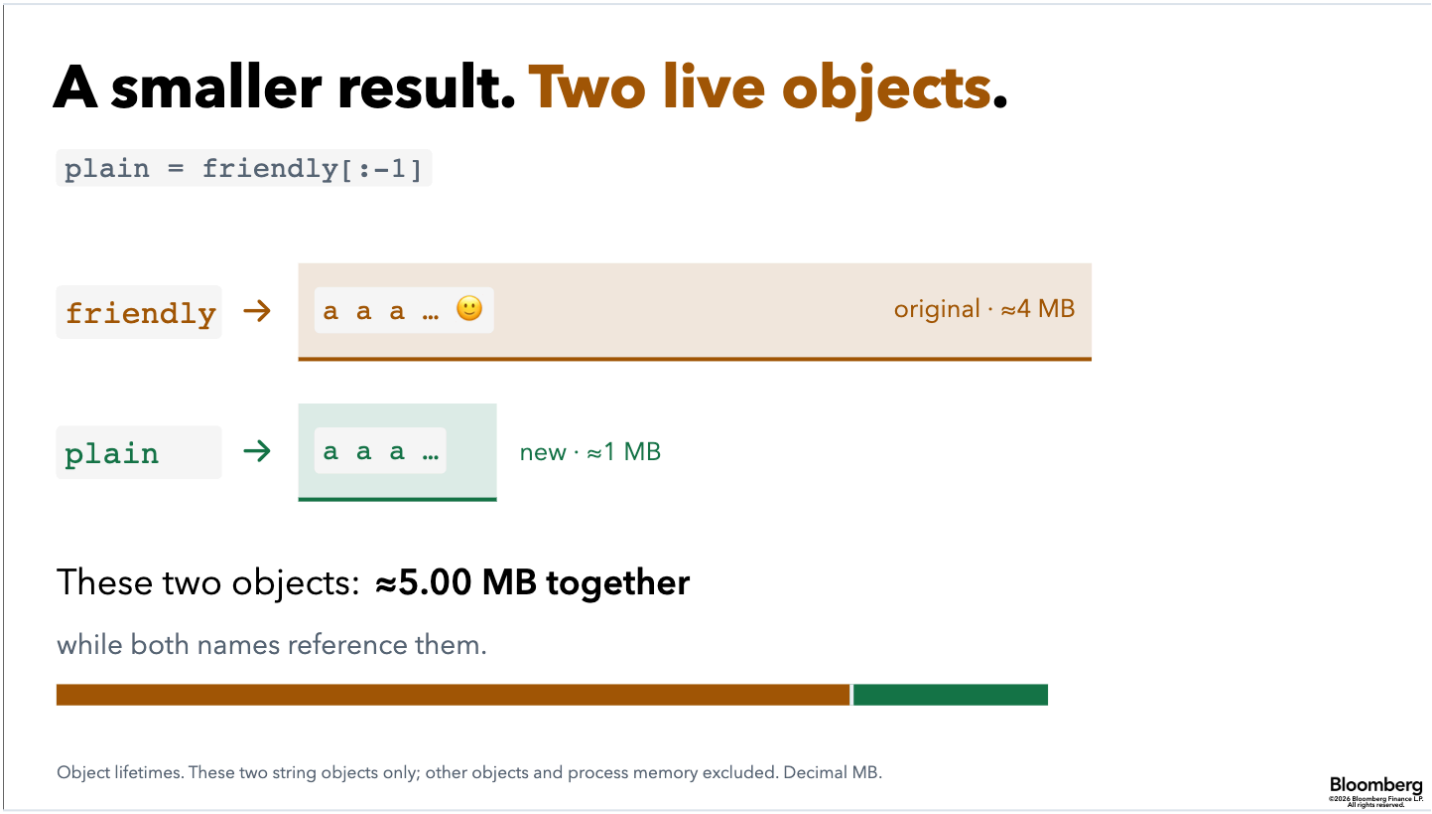
Bloomberg
©2024 Bloomberg Finance L.P.
All rights reserved.

[03:25–03:50]

Naturally, my first cost-saving proposal is to remove the happiness. Slice off the last character. The new string comes back at roughly one megabyte. Refund approved.

[Pause. Point to the new variable.]

Python strings are immutable. Slicing creates another string, and this one contains only "a" characters. CPython can choose one-byte slots again.




[03:50–04:10]

Friendly still refers to the original four-megabyte string.

[Click: characters copy into the smaller allocation. The original stays.]

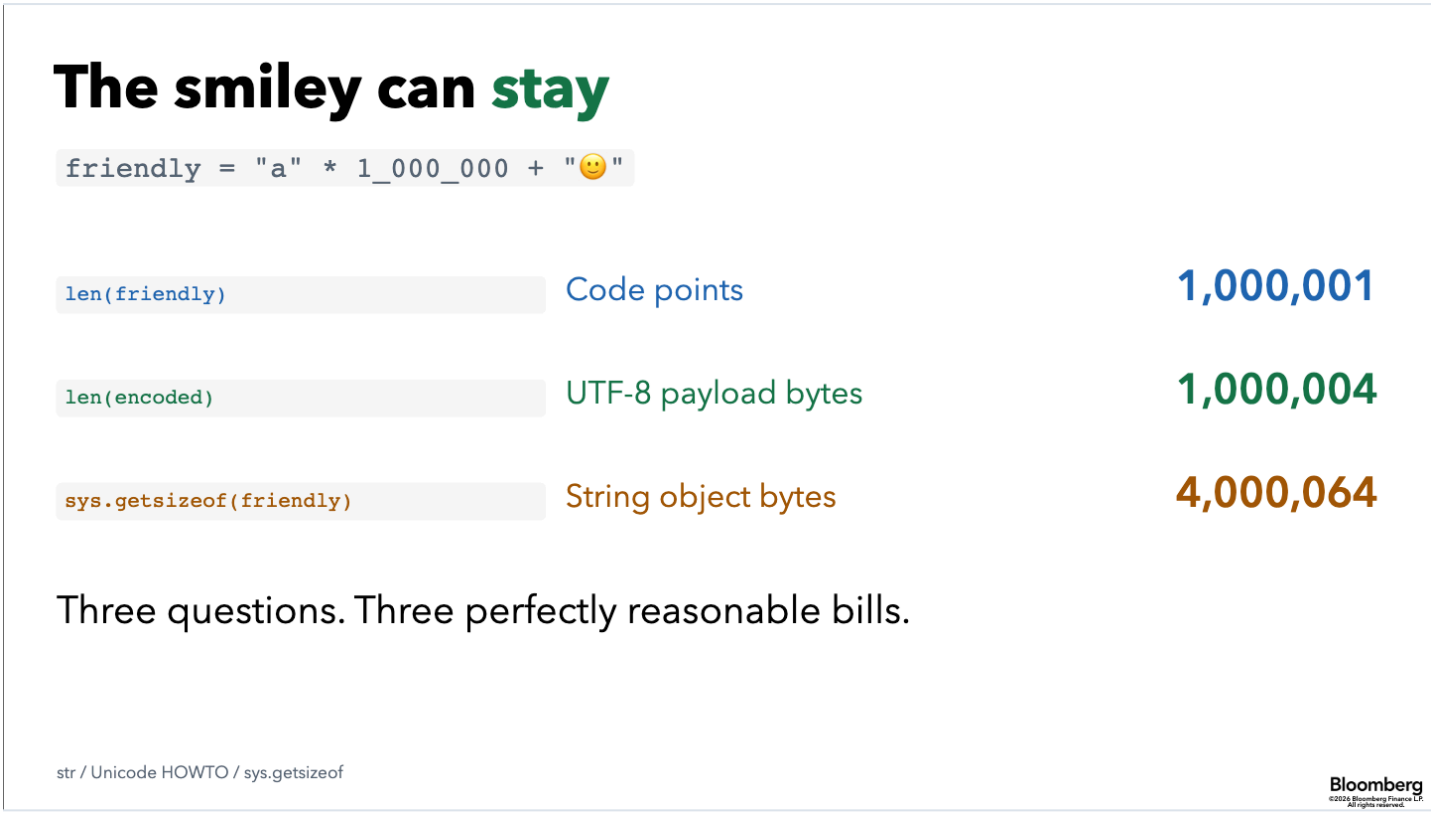
Plain refers to the new one-megabyte string.

[Click: the total rises from four to five megabytes.]

Five megabytes. My cost-saving proposal has cost us another megabyte.

[Pause on the total before explaining the lifetime.]

Both objects are still alive. The original can be reclaimed once nothing refers to it. Until then, producing a smaller result can mean a higher memory peak.




[04:10–04:25]

Please don't take this to work as a proposal to ban emojis. We have enough meetings already.

[Give the joke a beat before pointing to the first measurement.]

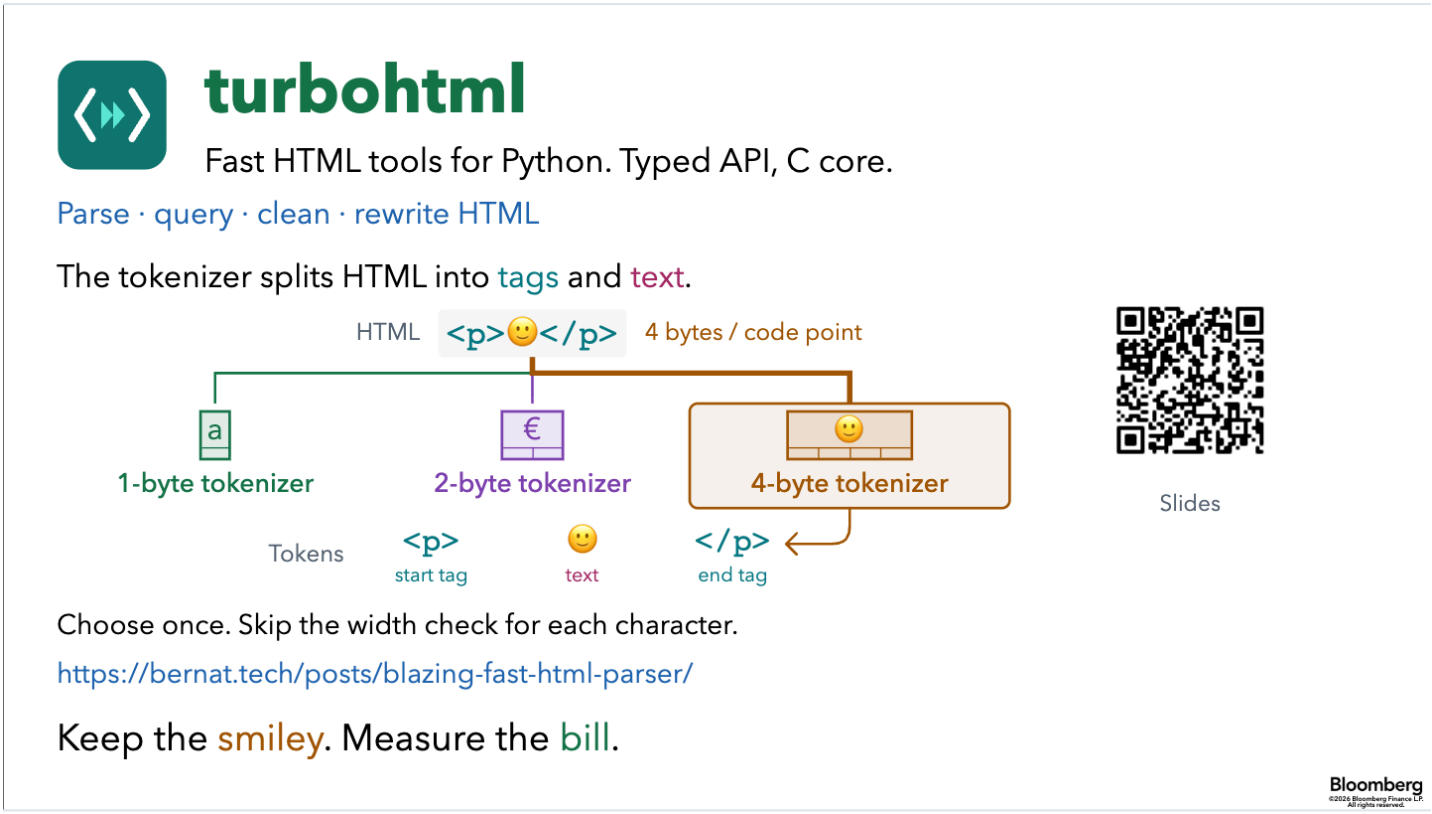
Length counts code points.

[Click: encoded payload.]

Encoding counts bytes.

[Click: string object.]

Get-size-of counts this object. Three questions. Three perfectly reasonable bills.




[04:25–04:45]

I use this in turbohtml, my fast HTML toolkit: parse, query, clean, and rewrite HTML from Python. Its tokenizer splits HTML into tags and text.

[Click: the four-byte string selects its tokenizer, which emits a start tag, text, and an end tag.]

The string's width selects a matching tokenizer once, avoiding the width check for every character.

[Click: article, slides QR code, and closing line.]

More string tricks are in the article. Keep the smiley. Measure the bill. Thank you.

[Sources: https://bernat.tech/posts/blazing-fast-html-parser/#one-machine-becomes-three and https://turbohtml.readthedocs.io/. Logo: tox-dev/turbohtml docs/_static/turbohtml.svg, MIT license, copied unchanged.]

[Leave the conclusion and QR code visible.]